04 · Insights: who should do what? (Task 3)

Transcript Intelligence: notebook 4 of 5

The task. Turn the categorized, scored calls into insights that specific stakeholders can act on, and show how each one is derived.

The answer in one line. `src/insights.py` builds one view per stakeholder from the outputs of `02_topics` and `03_sentiment`: a transparent **churn-risk score** per account (Sales / CS), the **blast radius** of the Detect outage (Engineering / execs), clustered **feature gaps** (Product), **competitor mentions** with the stated reason (Sales / Product), **talk time** vs call outcome (sales managers) and **action-item workload** (Ops / Support). `src/incidents.py` adds the incident register and monthly availability against the SLA.

Insight	Stakeholder	Built from
Account health / churn risk	Sales, Customer Success	confidence scores, churn-signal key moments, competitors, support cases, outage theme
Incident blast radius	Engineering, execs	outage-themed calls on one timeline from the first war room
Incident register and SLA	Engineering, Support	war rooms and support cases, with reported / owned / resolved times
Feature gaps	Product	`feature_gap` key moments, clustered and named
Competitive intelligence	Sales, Product	sentences naming a competitor, with the reason from the surrounding sentences
Talk time	Sales managers	customer share of speaking time per call vs the call's confidence score
Action items	Ops, Support	owners' workload and accounts whose commitments span several calls

How to run. `python -m src.pipeline` writes the insight tables to `data/processed/hybrid/` (Claude names the feature-gap clusters) and `data/processed/rules/` (no API calls). This notebook reads both. It makes no LLM calls. Only business calls are included: birthdays, farewells and other social calls are screened out before any of this is built (`01_explore`, section 4), so their action items, praise and competitor mentions are never counted. Numbers in the commentary were checked on 10/29/2026 against 115 calls; the tables and charts always reflect the current data. The same views, filtered by period, are in the dashboard's Accounts, Incidents and Insights tabs (`05_dashboard`).

1. Setup

In [1]:
```
%load_ext autoreload
%autoreload 2

import sys, json
from pathlib import Path

ROOT = Path.cwd().parent if Path.cwd().name == "notebooks" else Path.cwd()
sys.path.insert(0, str(ROOT))

import numpy as np
import pandas as pd
import plotly.express as px
import plotly.io as pio

from src.load import PROCESSED_DIR, mode_dir
from src.enrich import load_enriched, PERIODS, OUTAGE_START, OUTAGE_END
from src.topics import load_topics
from src.sentiment import load_sentiment
from src.insights import (RISK_WEIGHTS, RISK_TIERS, COMPETITOR_REASONS, OUTAGE_THEME,
                          load_insights, prepare_calls)
from src.incidents import SLA_TARGET, load_register, monthly_availability

pio.renderers.default = "plotly_mimetype+notebook_connected"
pd.set_option("display.max_columns", 50)
pd.set_option("display.max_colwidth", 140)
TYPE_COLORS = {"external": "#2a78d6", "support": "#eb6834", "internal": "#8a8f98"}
TIER_COLORS = {"High": "#d03b3b", "Medium": "#fab219", "Low": "#0ca30c"}
```

In [2]:
```
tables = load_enriched(PROCESSED_DIR)
hyb = load_insights(mode_dir("hybrid"))
rul = load_insights(mode_dir("rules"))
calls = prepare_calls(tables, load_topics(mode_dir("hybrid"))[0], load_sentiment()[1])
register = load_register(mode_dir("hybrid"))

health = hyb["account_health"]
print(f"{len(calls)} calls, {calls['account'].nunique()} customer accounts, "
```

```
      f"{calls['start_time'].min():%m/%d/%Y} – {calls['start_time'].max():%m/%d/%Y}")
print(json.dumps(hyb["summary"], indent=2, default=str))
```

115 calls, 33 customer accounts, 02/03/2026 – 07/01/2026


```
{
  "incident": {
    "first_war_room": "2026-03-10 13:15:00+00:00",
    "n_outage_calls": 51,
    "n_window_calls": 17,
    "n_affected_accounts": 24,
    "hours_to_first_customer_call": 0.0,
    "hours_to_last_window_call": 195.2,
    "window_avg_score": 2.18,
    "affected_avg_before": 3.66,
    "affected_avg_after": 3.42,
    "affected_naming_competitor_after": [
      "Atlas",
      "Brightpath",
      "Cobalt",
      "Ironclad",
      "Ironworks",
      "Northstar",
      "Nova",
      "Quantum",
      "Steelpoint",
      "Summit"
    ]
  },
  "n_accounts": 33,
  "risk_tiers": {
    "Medium": 15,
    "High": 10,
    "Low": 8
  },
  "top_risk_accounts": [
    "Summit",
    "Silverline",
    "Ridgeline",
    "Meridian",
    "Northstar"
  ],
  "n_feature_asks": 8,
  "competitor_calls": {
    "SentinelShield": 28,
```

```
    "VaultEdge": 16,
    "CyberNova": 8,
    "Fortiguard Pro": 5
  },
  "talk_share_vs_score_r": -0.121,
  "talk_bucket_scores": {
    "Q1 (33%\u201341%)": 3.58,
    "Q2 (41%\u201342%)": 3.82,
    "Q3 (42%\u201346%)": 3.14,
    "Q4 (46%\u201363%)": 3.15
  }
}
```


2. Account health: which accounts might we lose?

Stakeholder: Sales and Customer Success.

Why a weighted score, not a model

There are no churn outcomes in the data (no CRM, no renewals won or lost), so there's nothing to train a model on. A supervised model would also be a black box to the account manager who has to act on it. Instead each account gets a **transparent score out of 100**: six signals, each scaled 0–1 and multiplied by a fixed weight. Every point can be traced back to a listed driver.

In [3]:
```
signal_desc = {
    "churn_signals": "churn_signal key moments on the account's calls (full weight at 3+)",
    "competitor": "a competitor was named on any of the account's calls",
    "low_confidence": "latest call's confidence below 3.5 (full weight at 1.5)",
    "declining": "latest call at least 0.5 below the first",
    "support_load": "2 or more support cases",
    "outage_exposure": "had a call about the Detect outage",
}
pd.DataFrame([{"signal": k, "points": w, "what earns them": signal_desc[k]} for k, w in RISK_WEIGHTS.items()])
```

Out[3]:

	signal	points	what earns them
0	churn_signals	25	churn_signal key moments on the account's calls (full weight at 3+)
1	competitor	20	a competitor was named on any of the account's calls
2	low_confidence	20	latest call's confidence below 3.5 (full weight at 1.5)
3	declining	15	latest call at least 0.5 below the first
4	support_load	10	2 or more support cases
5	outage_exposure	10	had a call about the Detect outage

In [4]:

```
print("Tiers:", ", ".join(f"{t} ≥ {floor}" for floor, t in RISK_TIERS))
print(health["risk_tier"].value_counts().reindex(["High", "Medium", "Low"]).to_dict())
health.head(10)[["account", "risk_score", "risk_tier", "drivers", "n_calls", "avg_score", "last_score",
                 "competitors", "renewal_in_play"]]
```

```
Tiers: High ≥ 55, Medium ≥ 30, Low ≥ 0
{'High': 10, 'Medium': 15, 'Low': 8}
```

Out[4]:

	account	risk_score	risk_tier	drivers	n_calls	avg_score	last_score	competitors	renewal_in_play
0	Summit	88	High	4 churn signals; named VaultEdge; latest call 2.7/5; down 1.1 since first call; 2 support cases; hit by outage	4	2.83	2.7	[VaultEdge]	True
1	Silverline	76	High	3 churn signals; latest call 1.9/5; down 0.5 since first call; 4 support cases; hit by outage	5	3.08	1.9	[]	True
2	Ridgeline	71	High	3 churn signals; latest call 2.4/5; down 1.0 since first call; 2 support cases; hit by outage	3	2.63	2.4	[]	True
3	Meridian	66	High	3 churn signals; named SentinelShield; 2 support cases; hit by outage	3	3.17	3.4	[SentinelShield]	False
4	Northstar	65	High	4 churn signals; named CyberNova, SentinelShield; latest call 2.5/5; hit by outage	3	2.23	2.5	[CyberNova, SentinelShield]	True
5	Brightpath	65	High	4 churn signals; named SentinelShield; 2 support cases; hit by outage	5	3.34	3.7	[SentinelShield]	True
6	Nova	64	High	3 churn signals; named SentinelShield; latest call 2.6/5; hit by outage	2	2.45	2.6	[SentinelShield]	True
7	Axiom	60	High	2 churn signals; named SentinelShield, VaultEdge; latest call 2.7/5; down 1.5 since first call	2	3.45	2.7	[SentinelShield, VaultEdge]	True
8	Cobalt	56	High	3 churn signals; named SentinelShield; hit by outage	2	2.60	3.4	[SentinelShield]	True
9	Coastal	55	High	3 churn signals; named CyberNova, VaultEdge; 2 support cases	4	3.53	4.8	[CyberNova, VaultEdge]	False

Where the points come from

The score is rebuilt below from the account columns, signal by signal, so the stacked bar shows *why* each account ranks where it does.

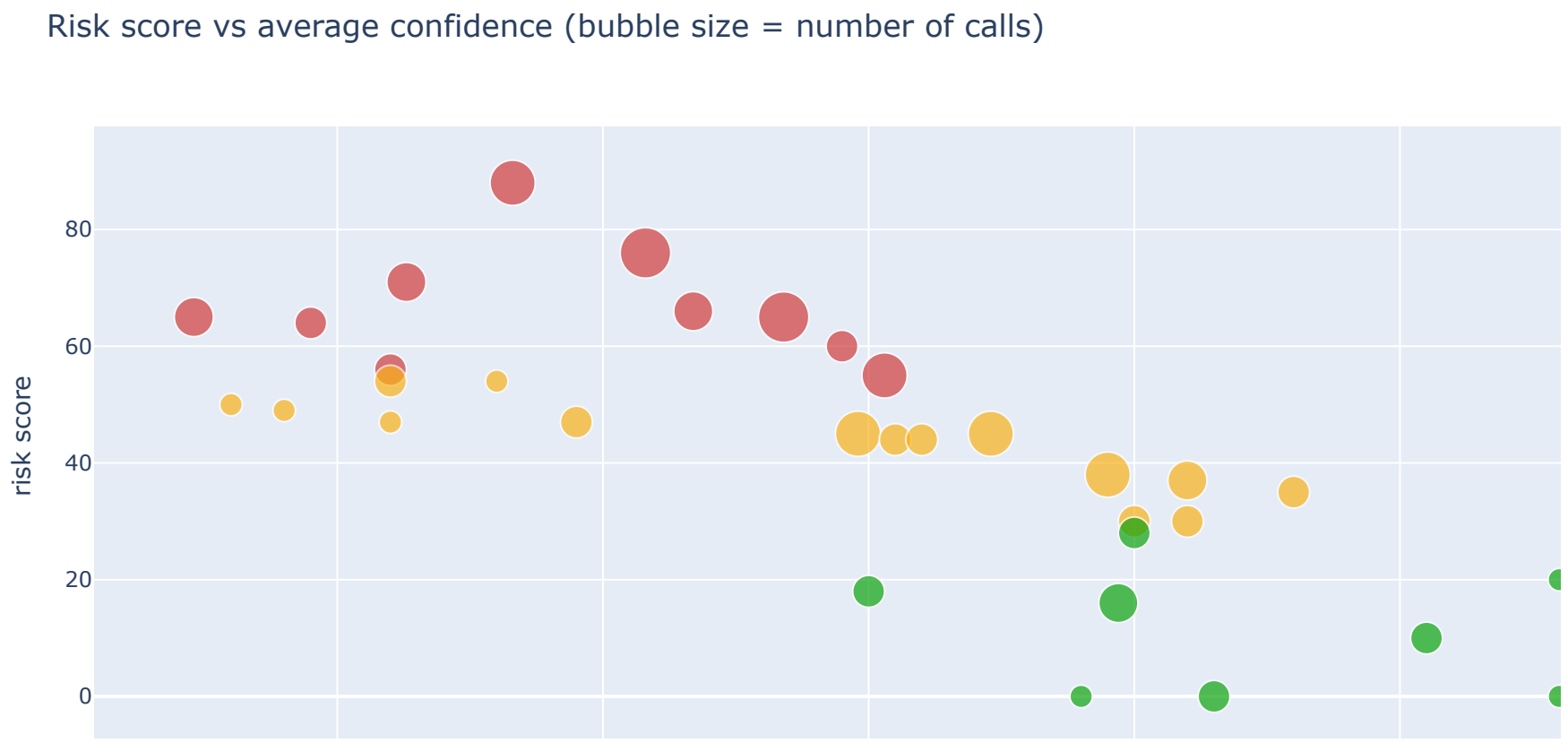
In [5]:

```
parts = pd.DataFrame({
    "churn_signals": health["churn_signals"].clip(upper=3) / 3,
    "competitor": health["competitors"].map(len).gt(0).astype(float),
    "low_confidence": ((3.5 - health["last_score"]) / 2).clip(0, 1),
    "declining": (health["score_trend"] <= -0.5).astype(float),
    "support_load": (health["n_support"] >= 2).astype(float),
    "outage_exposure": health["outage_affected"].astype(float),
}) * pd.Series(RISK_WEIGHTS)
parts.index = health["account"]
assert (parts.sum(axis=1).round().astype(int).values == health["risk_score"].values).all(), "score rebuild mismatch"

top = parts.head(15).reset_index().melt(id_vars="account", var_name="signal", value_name="points")
fig = px.bar(top, y="account", x="points", color="signal", orientation="h",
             category_orders={"account": health.head(15)["account"].tolist(), "signal": list(RISK_WEIGHTS)},
             color_discrete_sequence=["#2a78d6", "#eb6834", "#7a5cd6", "#1c9e8a", "#c9579b", "#8a8f98"],
             title="Risk score by signal, 15 highest-risk accounts")
fig.add_vline(x=RISK_TIERS[0][0], line_dash="dot", line_color=TIER_COLORS["High"],
              annotation_text="High", annotation_position="top")
fig.update_layout(yaxis_title=None, xaxis_title="points out of 100", bargap=0.25)
fig
```



In [6]:
```
px.scatter(health, x="avg_score", y="risk_score", color="risk_tier", size="n_calls",
           color_discrete_map=TIER_COLORS, hover_name="account", hover_data=["drivers"],
           category_orders={"risk_tier": ["High", "Medium", "Low"]},
           labels={"avg_score": "average confidence across the account's calls (1-5)", "risk_score": "risk score"},
           title="Risk score vs average confidence (bubble size = number of calls)")
```



What this shows.

- **Every high-risk account combines several signals.** No account reaches *High* on one bad call: the leaders (Summit, Silverline, Ridgeline, Meridian, Northstar, Brightpath) each pair repeated churn signals with outage exposure, and most also have a named competitor, a falling score or a pile of support cases. That makes the list hard to dismiss as noise.
- **Risk is not the same as a low average.** Some accounts with a respectable average are still high risk because their *latest* call is the worst one, or because they named a competitor. A dashboard that only ranked by average confidence would miss them.

- **Renewal is in play for most of the top accounts**, so the timing matters: the account brief in the dashboard (Accounts tab → click an account) lists the open action items to work through.

Is the ranking stable?

The rules-only run uses keyword themes instead of Claude's, which changes the competitor and outage inputs slightly. If the tiers barely move, the ranking doesn't depend on the LLM.

In [7]:
```
cmp = health[["account", "risk_score", "risk_tier"]].merge(
    rul["account_health"][["account", "risk_score", "risk_tier"]], on="account", suffixes=("_hybrid", "_rules"))
print(f"Same tier in both modes: {(cmp['risk_tier_hybrid'] == cmp['risk_tier_rules']).mean():.0%} of accounts; "
      f"rank correlation of scores: {cmp['risk_score_hybrid'].corr(cmp['risk_score_rules'], method='spearman'):.2f}")
cmp[cmp["risk_tier_hybrid"] != cmp["risk_tier_rules"]]
```

```
Same tier in both modes: 97% of accounts; rank correlation of scores: 0.99
```

Out[7]:

	account	risk_score_hybrid	risk_tier_hybrid	risk_score_rules	risk_tier_rules
26	Larkspur	20	Low	30	Medium

Sanity check against what happened next. For accounts with calls after the outage, does the score agree with how their *last* call went? A high score should go with a poor latest call, but the score shouldn't simply *be* the latest call, or the other five signals would add nothing.

In [8]:
```
print(f"Correlation of risk score with latest call confidence: {health['risk_score'].corr(health['last_score']):.2f}")
health.groupby("risk_tier")[["last_score", "avg_score", "churn_signals", "n_support"]].mean().round(2) \
      .reindex(["High", "Medium", "Low"])
```

```
Correlation of risk score with latest call confidence: -0.64
```

Out[8]:

risk_tier	last_score	avg_score	churn_signals	n_support
High	3.01	2.93	3.20	1.70
Medium	3.45	3.36	1.40	0.80
Low	4.35	4.21	0.25	0.38

3. Incident blast radius: what did the outage reach?

Stakeholder: Engineering leadership and execs.

Every call tagged with the *Detect outage & fallout* theme from the outage start on is placed on one timeline, measured in hours from the first internal war room. That shows how fast the incident reached customers, how long it kept generating calls, and which accounts it touched.

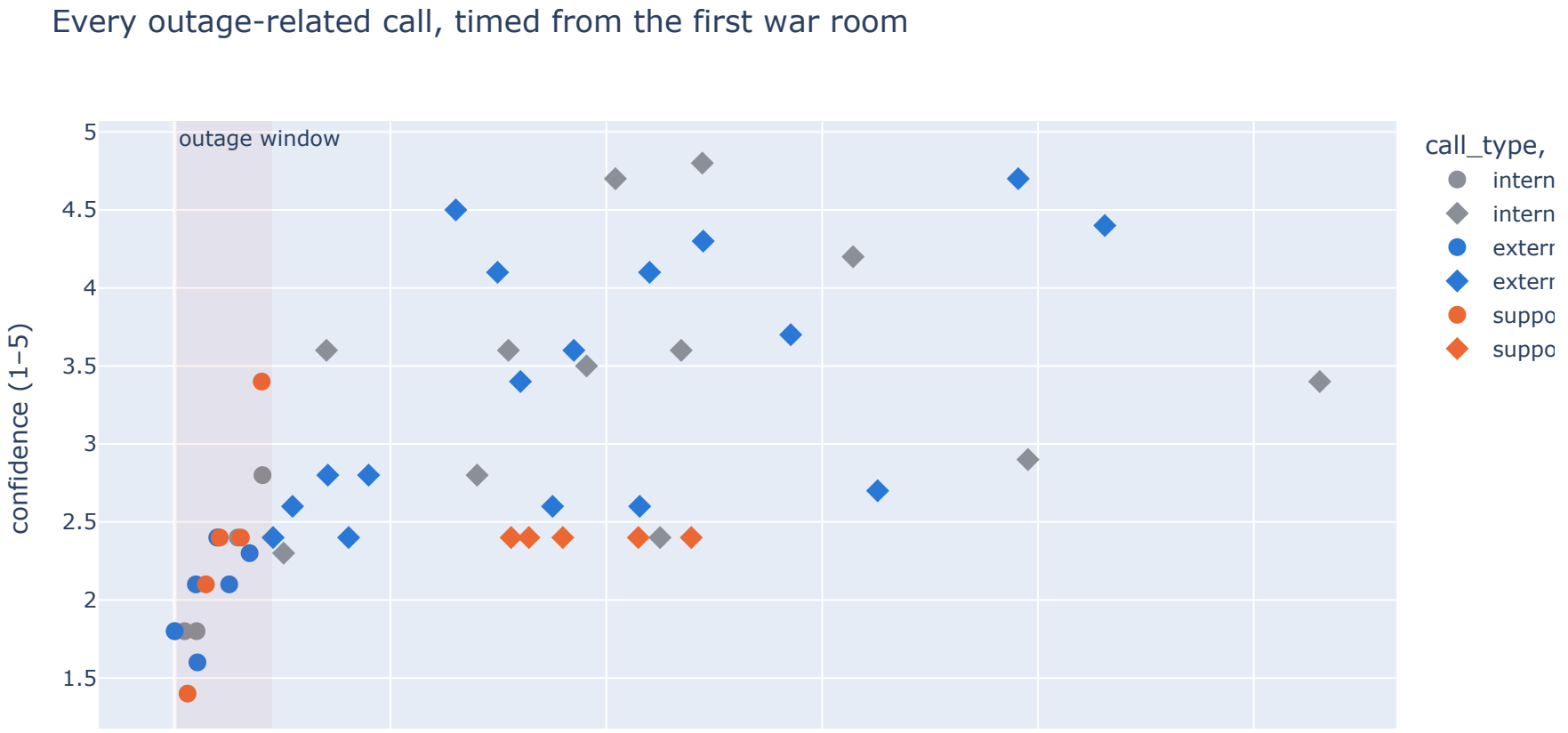
In [9]:
```
tl = hyb["incident_timeline"]
inc = hyb["summary"]["incident"]
pd.Series({
    "first war room": pd.Timestamp(inc["first_war_room"]).strftime("%m/%d/%Y %H:%M"),
    "outage-related calls (from outage start on)": inc["n_outage_calls"],
    "customer accounts affected": inc["n_affected_accounts"],
    "hours to first customer call": inc["hours_to_first_customer_call"],
    "average confidence during the outage window": inc["window_avg_score"],
    "affected accounts, average before → after": f"{inc['affected_avg_before']} → {inc['affected_avg_after']}",
    "affected accounts naming a competitor afterwards": len(inc["affected_naming_competitor_after"]),
}).to_frame("value")
```

Out[9]:

	value
first war room	03/10/2026 13:15
outage-related calls (from outage start on)	51
customer accounts affected	24
hours to first customer call	0.0
average confidence during the outage window	2.18
affected accounts, average before → after	3.66 → 3.42
affected accounts naming a competitor afterwards	10

In [10]:
```
plot = tl.assign(days=tl["hours_from_start"] / 24, account=tl["account"].fillna("(internal)"))
fig = px.scatter(plot, x="days", y="sentiment_score", color="call_type", color_discrete_map=TYPE_COLORS,
                 symbol="in_window", hover_name="title", hover_data=["account", "competitors"],
```

```
                labels={"days": "days since the first war room", "sentiment_score": "confidence (1–5)",
                        "in_window": "during outage window"},
                title="Every outage-related call, timed from the first war room")
window_days = (pd.Timestamp(OUTAGE_END) - pd.Timestamp(OUTAGE_START)).days + 1
fig.add_vrect(x0=0, x1=window_days, fillcolor="#d03b3b", opacity=0.06, line_width=0,
              annotation_text="outage window", annotation_position="top left")
fig.update_traces(marker_size=10)
fig
```



In [11]:
```
# Affected accounts' confidence by period: did they recover?
affected = set(tl.loc[tl["call_type"] != "internal", "account"].dropna())
by_period = (calls[calls["account"].isin(affected)]
             .groupby("period")["sentiment_score"].agg(["mean", "count"]).round(2).reindex(PERIODS))
others = calls[calls["account"].notna() & ~calls["account"].isin(affected)] \
             .groupby("period")["sentiment_score"].mean().round(2).reindex(PERIODS)
by_period.assign(other_accounts=others).rename(columns={"mean": "affected_accounts", "count": "affected_calls"})
```

Out[11]:

period	affected_accounts	affected_calls	other_accounts
Pre-outage	3.66	16	3.99
Outage	2.18	11	NaN
Recovery	3.29	10	3.50
Post-Comply-launch	3.48	25	4.06

What this shows.

- **Customers knew immediately.** The first customer escalation landed in the same hour as the internal war room, so customers found the outage at the same time as engineering, not after being told. Proactive detection and notification is a gap (it also comes up as a feature ask in section 5).
- **The outage generated calls long after it was fixed.** About a third of outage-related calls fall inside the 10–18 March window; the rest are fallout: SLA credit requests, renewal conversations and competitive evaluations weeks later.
- **Affected accounts haven't fully recovered.** Their average confidence is still below where it was before the outage, and several of them named a competitor afterwards. Those accounts are the overlap between this section and the risk list in section 2.

Incident register and the SLA

`src/incidents.py` turns war-room calls and support cases into a register with reported, owned and resolved times, and works out downtime per month against the SLA.

In [12]:
```
print(register["kind"].value_counts().to_dict(), "|", register["status"].value_counts().to_dict())
show = register.assign(reported=register["reported_at"].dt.strftime("%m/%d/%Y %H:%M"),
                       resolved=register["resolved_at"].dt.strftime("%m/%d/%Y %H:%M"))
show[["incident_id", "kind", "title", "account", "category", "priority", "status", "owner", "owner_team",
      "reported", "minutes_to_own", "resolved", "hours_to_resolve"]].head(15)
```

```
{'Support case': 32, 'Major incident': 1} | {'Open': 26, 'Resolved': 7}
```

Out[12]:

	incident_id	kind	title	account	category	priority	status	owner	owner_team	reported	minutes_to_own	re
0	CASE-3286	Support case	Vanta Health Systems API Integration Error	Vanta	Outage	P1	Open	Priya Patel	Support	02/03/2026 10:00	0.1	
1	CASE-9279	Support case	Summit Trust Billing Inquiry	Summit	Billing	P3	Open	Sarah Chen	Support	02/06/2026 09:15	0.1	
2	CASE-3615	Support case	Nimbus Platform Custom Report Request	Nimbus	Request	P3	Open	David Kim	Support	02/10/2026 15:30	0.1	
3	CASE-6635	Support case	Nova Retail Group SSO Login Failures	Nova	Outage	P1	Open	Elena Vasquez	Support	02/14/2026 13:00	0.7	
4	CASE-1750	Support case	Vanta Health Systems SIEM Connector Issue	Vanta	Defect	P1	Open	Sarah Chen	Support	02/18/2026 12:30	0.1	
5	CASE-6977	Support case	Brightpath Commerce Slow Backup Performance	Brightpath	Degradation	P2	Open	David Kim	Support	02/21/2026 11:30	0.1	
6	CASE-3677	Support case	Coastal Living Co MFA Token Failures	Coastal	Outage	P1	Open	Elena Vasquez	Support	02/24/2026 16:45	0.1	
7	CASE-7572	Support case	Brightpath Commerce	Brightpath	Billing	P3	Open	Marcus Williams	Support	02/27/2026 13:00	0.3	

	incident_id	kind	title	account	category	priority	status	owner	owner_team	reported	minutes_to_own	re
			Invoice Discrepancy									
8	CASE-2638	Support case	Pineridge Systems CloudPrime S3 Backup Connector	Pineridge	Defect	P2	Open	Sarah Chen	Support	03/04/2026 10:15	1.1	
9	CASE-3546	Support case	Meridian Capital SCIM Provisioning Request	Meridian	Request	P3	Open	Sarah Chen	Support	03/07/2026 13:30	0.1	
10	INC-20260310	Major incident	Detect Pipeline Failure	Blackridge, Cobalt, Helix, Meridian, Northstar, Quantum	Outage	P1	Resolved	Tom Bradley	Engineering	03/10/2026 13:15	0.4	03/1
11	CASE-3266	Support case	Trailhead Marketplace Detect Alerts Not Firing	Trailhead	Outage	P1	Resolved	Elena Vasquez	Support	03/11/2026 17:45	0.1	03/1
12	CASE-6573	Support case	Ridgeline Logistics Detect Data Gaps	Ridgeline	Outage	P1	Resolved	Elena Vasquez	Support	03/13/2026 10:30	0.1	03/1
13	CASE-5889	Support case	Ridgeline Logistics Detect Latency Issues	Ridgeline	Degradation	P2	Resolved	Priya Patel	Support	03/14/2026 17:15	0.1	03/1

	incident_id	kind	title	account	category	priority	status	owner	owner_team	reported	minutes_to_own	re
14	CASE-3103	Support case	Silverline Brands Detect False Positives After Patch	Silverline	Defect	P2	Resolved	Sarah Chen	Support	03/16/2026 16:15	0.1	03/1

In [13]:
```
avail = monthly_availability(register, calls["start_time"].min(), calls["start_time"].max(), SLA_TARGET)
print(f"SLA target {SLA_TARGET:.1%} uptime")
avail
```

```
SLA target 99.5% uptime
```

Out[13]:

	month	label	outage_hours	n_outages	pct_time_down	availability	allowed_hours	sla_target	sla_breached	hours_over_sla	incidents
0	2026-02-01	Feb 2026	0.0	0	0.000000	1.000000	3.36	0.995	False	0.00	
1	2026-03-01	Mar 2026	6.0	1	0.008065	0.991935	3.72	0.995	True	2.28	INC-20260310
2	2026-04-01	Apr 2026	0.0	0	0.000000	1.000000	3.60	0.995	False	0.00	
3	2026-05-01	May 2026	0.0	0	0.000000	1.000000	3.72	0.995	False	0.00	
4	2026-06-01	Jun 2026	0.0	0	0.000000	1.000000	3.60	0.995	False	0.00	
5	2026-07-01	Jul 2026	0.0	0	0.000000	1.000000	3.72	0.995	False	0.00	

In [14]:
```
(register.groupby(["category", "status"]).size().unstack(fill_value=0)
         .assign(total=lambda d: d.sum(axis=1)).sort_values("total", ascending=False))
```

Out[14]:

status category	Open	Resolved	total
Defect	8	2	10
Outage	3	4	7
Degradation	5	1	6
Billing	5	0	5
Request	5	0	5

What this shows. March is the only month over the SLA: the Detect outage alone used more downtime than the month allows at 99.5%, which is why SLA credits come up on customer and support calls. Most support cases are still open in the data. That's partly real (the follow-up hasn't happened yet) and partly a limit of the method: a case only counts as resolved if a call says so, because there's no ticketing system to confirm it.

4. Feature gaps: what are customers asking for?

Stakeholder: Product.

The vendor tags some key moments as `feature_gap`. Individually they're anecdotes; clustered, they become a ranked list of asks. Each moment is embedded, grouped with KMeans (k = 8), and each group is named. In **Hybrid** mode Claude names the group from its moments; in **Rules only** mode the name is the group's top TF-IDF terms. Asks are ranked by the number of **distinct accounts** raising them, not mentions, so one vocal customer can't top the list.

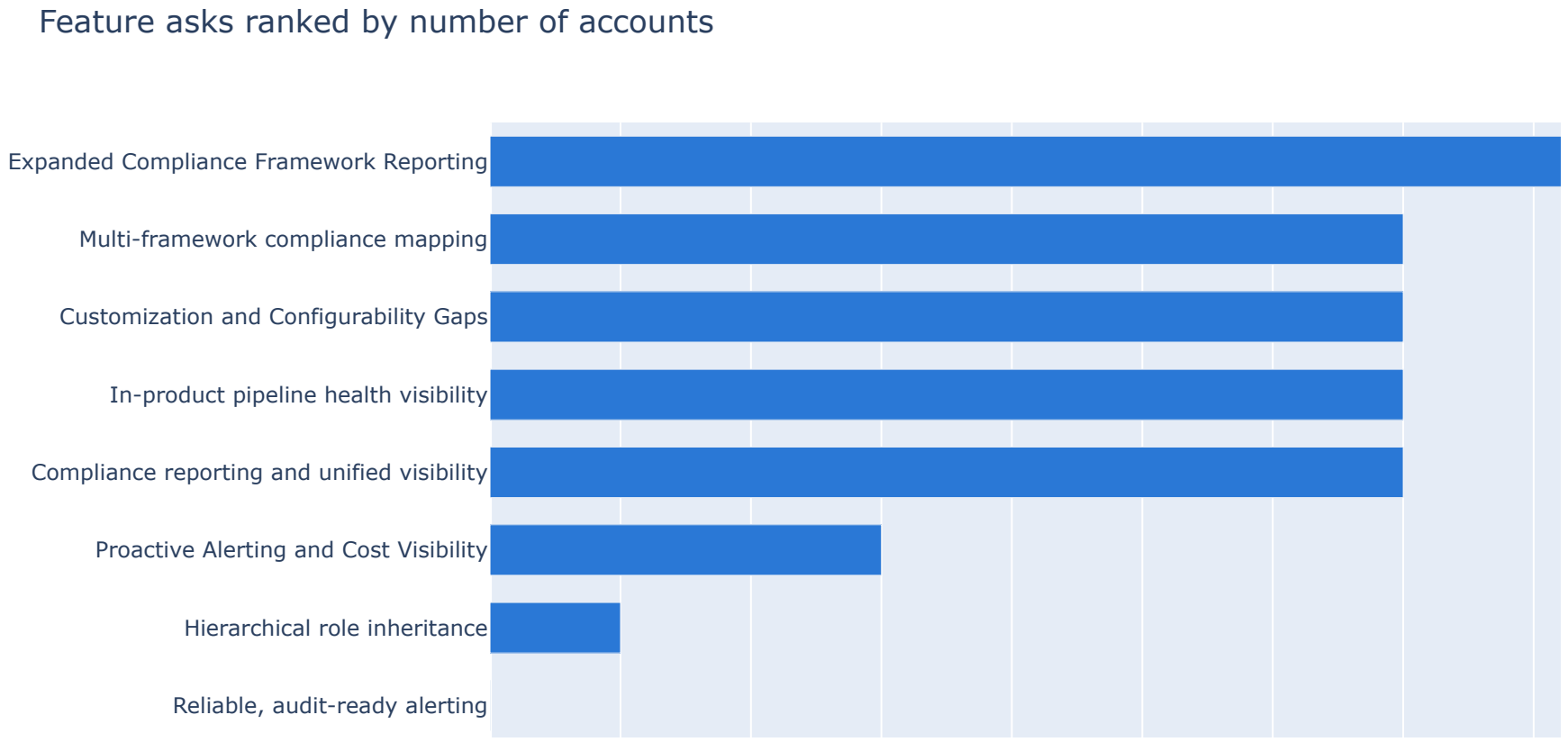
In [15]:
```
asks = hyb["feature_gaps"]
asks[["ask", "n_accounts", "n_mentions", "accounts", "summary"]]
```

Out[15]:

	ask	n_accounts	n_mentions	accounts	summary
0	Expanded Compliance Framework Reporting	9	14	[Atlas, Blackridge, Crestline, Larkspur, Nimbus, Steelpoint, Stratos, Trailhead, Vanta]	Customers want Aegis Comply to offer on-demand, multi-framework compliance reporting that covers more frameworks (SOC 2, CMMC, NCUA, GDP...
1	Compliance reporting and unified visibility	7	12	[Atlas, Blackridge, Bridgeport, Coastal, Meridian, Pineridge, Redwood]	Customers want filterable, unified compliance reports and dashboards (HIPAA and SOC 2 together, by control family) plus remediation guid...
2	In-product pipeline health visibility	7	10	[Axiom, Blackridge, Brightpath, Cobalt, Crestline, Frostbyte, Pinnacle]	Customers want the console to show real-time health, sync latency, and outage status of the monitoring and event ingestion pipeline so t...
3	Customization and Configurability Gaps	7	8	[Axiom, Blackridge, Forge, Helix, Ridgeline, Steelpoint, Summit]	Customers want more control and flexibility over the product, including custom control labels and compliance templates, easier rule conf...
4	Multi-framework compliance mapping	7	7	[Coastal, Keystone, Maplewood, Meridian, Nimbus, Silverline, Summit]	Customers want compliance reporting that maps controls across SOC 2, ISO 27001, and PCI DSS in a true matrix format, along with PCI DSS ...
5	Proactive Alerting and Cost Visibility	3	5	[Nova, Summit, Vanta]	Customers want proactive alerts and automation, such as seat-count overage warnings, coverage-gap notifications for unprotected VMs, and...
6	Hierarchical role inheritance	1	3	[Frostbyte]	Customers want nested role inheritance, plus per-role or per-resource session timeout policies, so they can stop duplicating flat roles ...
7	Reliable, audit-ready alerting	0	4	[]	Customers want trustworthy threat-monitoring alerts that reduce alert fatigue, proactively warn when telemetry stops arriving, hold up t...

In [16]:

```
fig = px.bar(asks.sort_values("n_accounts"), y="ask", x="n_accounts", orientation="h",
             hover_data=["n_mentions", "summary"], color_discrete_sequence=["#2a78d6"],
             labels={"n_accounts": "accounts asking", "ask": ""},
             title="Feature asks ranked by number of accounts")
fig.update_layout(bargap=0.35)
fig
```



In [17]:
```
# Naming: Claude vs top terms, same clusters
pd.DataFrame({"Hybrid (Claude)": asks.sort_values("gap_cluster")["ask"].values,
              "Rules only (top terms)": rul["feature_gaps"].sort_values("gap_cluster")["ask"].values})
```

Out[17]:

	Hybrid (Claude)	Rules only (top terms)
0	Customization and Configurability Gaps	Requests – internal, custom
1	Expanded Compliance Framework Reporting	Comply – v2, framework
2	Proactive Alerting and Cost Visibility	Raymond – support, proactive
3	Multi-framework compliance mapping	Confirms – soc, kevin
4	Reliable, audit-ready alerting	Alert – risk, day
5	Compliance reporting and unified visibility	Identifies – point, gap
6	In-product pipeline health visibility	Visibility – asks, product
7	Hierarchical role inheritance	Role – victor, inheritance

In [18]:

```
# Read the moments behind any ask: change ASK to a name from the table above
ASK = asks["ask"].iloc[0]
hyb["feature_gap_moments"].query("ask == @ASK")[["account", "speaker", "period", "text"]].reset_index(drop=True)
```

Out[18]:

	account	speaker	period	text
0	Atlas	Wayne Keller	Pre-outage	Wayne asks about SOC 2 compliance reporting, signaling potential expansion opportunity with the Aegis Comply add-on module
1	Stratos	James Mitchell	Pre-outage	James reveals that on-demand reporting is not yet available but is coming in an upcoming release
2	Trailhead	Isaac Brennan	Recovery	Isaac reacts positively to Aegis Comply v2's on-demand PCI DSS reporting, noting his compliance team spends six weeks in spreadsheets fo...
3	Steelpoint	Harold Jensen	Recovery	Harold flags that CMMC framework is not covered by the upcoming Comply v2 release, which is a gap given his Q4 audit
4	Vanta	Dr. Linda Cheng	Post-Comply-launch	Linda requests multi-region replication for a new Sacramento site
5	NaN	Megan Lawson	Post-Comply-launch	Megan says Protect multi-region replication has slipped to early Q4
6	Crestline	Derek Owens	Post-Comply-launch	Derek raises interest in Aegis Comply v2 after seeing the product announcement, opening a potential upsell opportunity
7	NaN	Mike Romano	Post-Comply-launch	Mike says multi-region replication will likely slip to early Q4
8	NaN	Chris Lee	Post-Comply-launch	Chris Lee connects Comply v2's new on-demand HIPAA reporting as a potential solution for Treswick Healthcare's regulatory documentation ...
9	Larkspur	Lisa Park	Post-Comply-launch	Lisa notes the NCUA examination framework is not built into Comply v2
10	Nimbus	Keith Donovan	Pre-outage	Keith describes the core feature gap: no native cross-framework control mapping report exists in Aegis Comply
11	NaN	Jordan Whitfield	Post-Comply-launch	Jordan raises GDPR as the next framework to add, with Diana confirming strong demand from the European segment since Q3 last year
12	NaN	Sofia Petrov	Post-Comply-launch	Sofia estimates the scheduler refactor will take six to eight weeks minimum, creating a critical timing conflict with the active Protect...
13	Blackridge	Julia Tran	Pre-outage	Julia expresses immediate excitement about Aegis Comply v2 on-demand multi-framework reporting and asks to be included in early access

In [19]:
```
# The asks re-cluster on every run, so the headline is computed here rather than written down
high = set(health.loc[health["risk_tier"] == "High", "account"])
for label, pattern in [("compliance reporting / framework mapping", "complian|framework"),
                       ("visibility / alerting", "visib|alert")]:
    g = asks[asks["ask"].str.contains(pattern, case=False)]
    acc = set().union(*g["accounts"].map(set)) if len(g) else set()
    print(f"{label}: {len(g)} of {len(asks)} groups, {len(acc)} of {health['account'].nunique()} accounts; "
          f"high-risk among them: {', '.join(sorted(acc & high)) or 'none'}")
```

```
compliance reporting / framework mapping: 3 of 8 groups, 18 of 33 accounts; high-risk among them: Coastal, Meridian, Silverlin
e, Summit
visibility / alerting: 4 of 8 groups, 16 of 33 accounts; high-risk among them: Axiom, Brightpath, Coastal, Cobalt, Meridian, No
va, Summit
```

What this shows.

- **Compliance across frameworks dominates.** The top ask and two others are about reporting or control mapping across SOC 2, ISO 27001, PCI DSS, HIPAA and others. Together they come from more than half of all customer accounts (18 of 33 when this was written), including four high-risk ones (Coastal, Meridian, Silverline, Summit). That's direct evidence for Comply v2's roadmap.
- **Visibility is the second theme.** Several groups ask to *see problems sooner*: in-product pipeline health, proactive alerting and unified visibility. That's the outage lesson from section 3 (customers found the outage at the same time as engineering), coming back as product asks.
- **Claude's names are what makes the list usable.** The clusters are identical in both modes, but the top-terms names are often a verb or a person's first name, which means nothing to a product manager. Claude's names can go straight onto a roadmap slide. This is the only LLM step in the insights module, and it's cached.
- **The groups move as data arrives.** Clustering is re-run on every pipeline run, so when new transcripts add moments the groups, their names and their counts can shift. For example, an earlier run had *hierarchical role inheritance* as a five-account ask; it's now a single account, with the rest absorbed into broader groups. Quote the moments, not just the counts.
- **Clustering isn't perfect.** A group's example quote sometimes belongs to a neighbouring ask (KMeans assigns every moment somewhere), so read the moments before quoting a number externally.

5. Competitive intelligence: who are we losing to, and why?

Stakeholder: Sales and Product.

Every sentence naming a competitor is kept, with who said it (customer or AegisCloud) and the reason. The reason is matched in that sentence **and its neighbours**, since "we're looking at SentinelShield" often has the *why* in the next sentence.

In [20]:
```
pd.DataFrame([{"reason": k, "matches": p.pattern} for k, p in COMPETITOR_REASONS.items()])
```

Out[20]:

	reason	matches
0	Pricing	pric|cheaper|discount|cost|budget|quote
1	Reliability / uptime	uptime|SLA|outage|reliab|redundan|downtime|99\.9
2	Features	feature|capabilit|integrat|report|dashboard|coverage
3	Outreach	reached out|approached|contacted|reach out|pitch

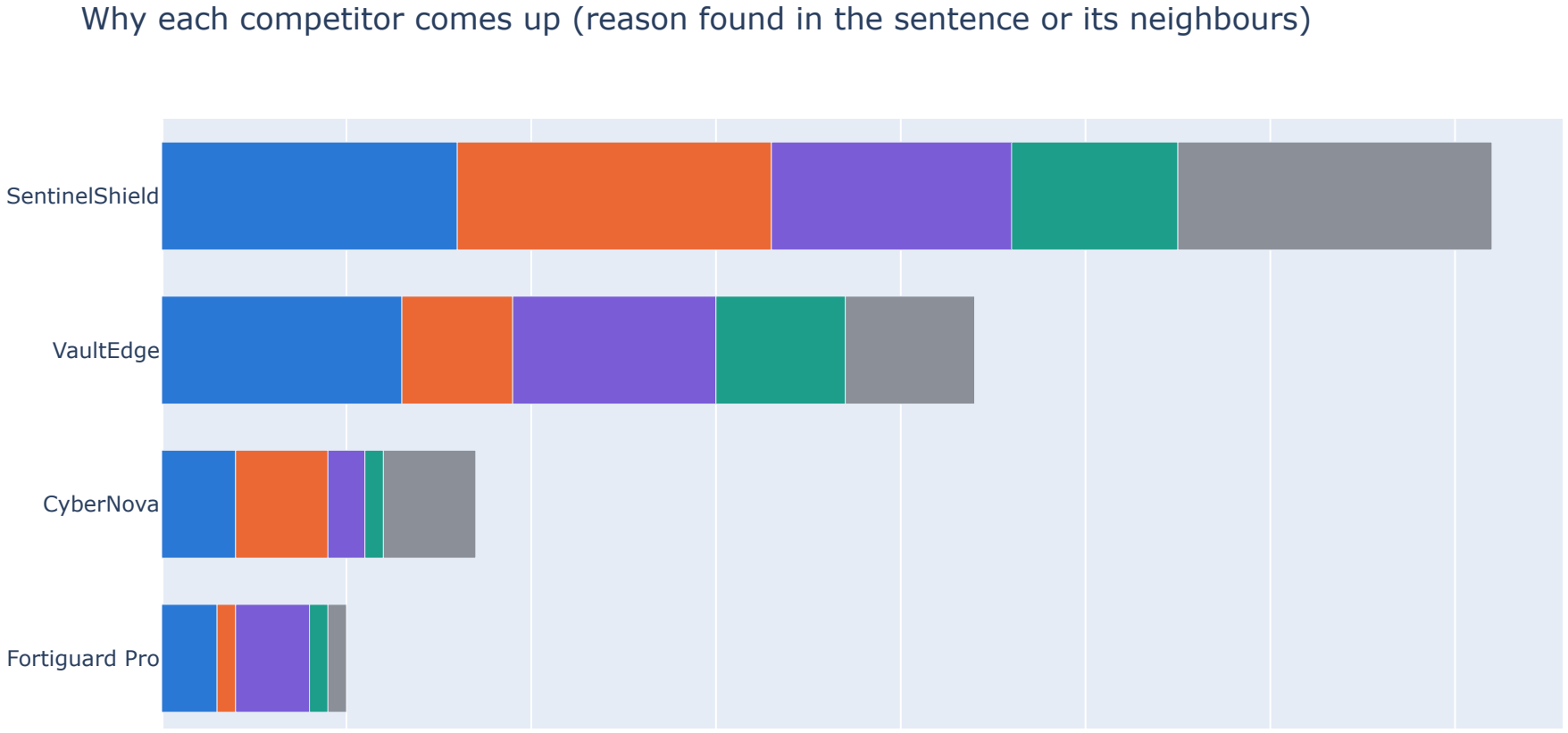
In [21]:
```
comp = hyb["competitor_mentions"]
summary = (comp.groupby("competitor")
               .agg(calls=("call_id", "nunique"), mentions=("quote", "size"),
                    accounts=("account", lambda a: a.nunique()),
                    customer_said=("speaker_side", lambda s: (s == "customer").mean()))
               .round(2).sort_values("calls", ascending=False))
summary
```

Out[21]:

competitor	calls	mentions	accounts	customer_said
SentinelShield	28	57	13	0.37
VaultEdge	16	35	7	0.31
CyberNova	8	16	3	0.19
Fortiguard Pro	5	8	1	0.12

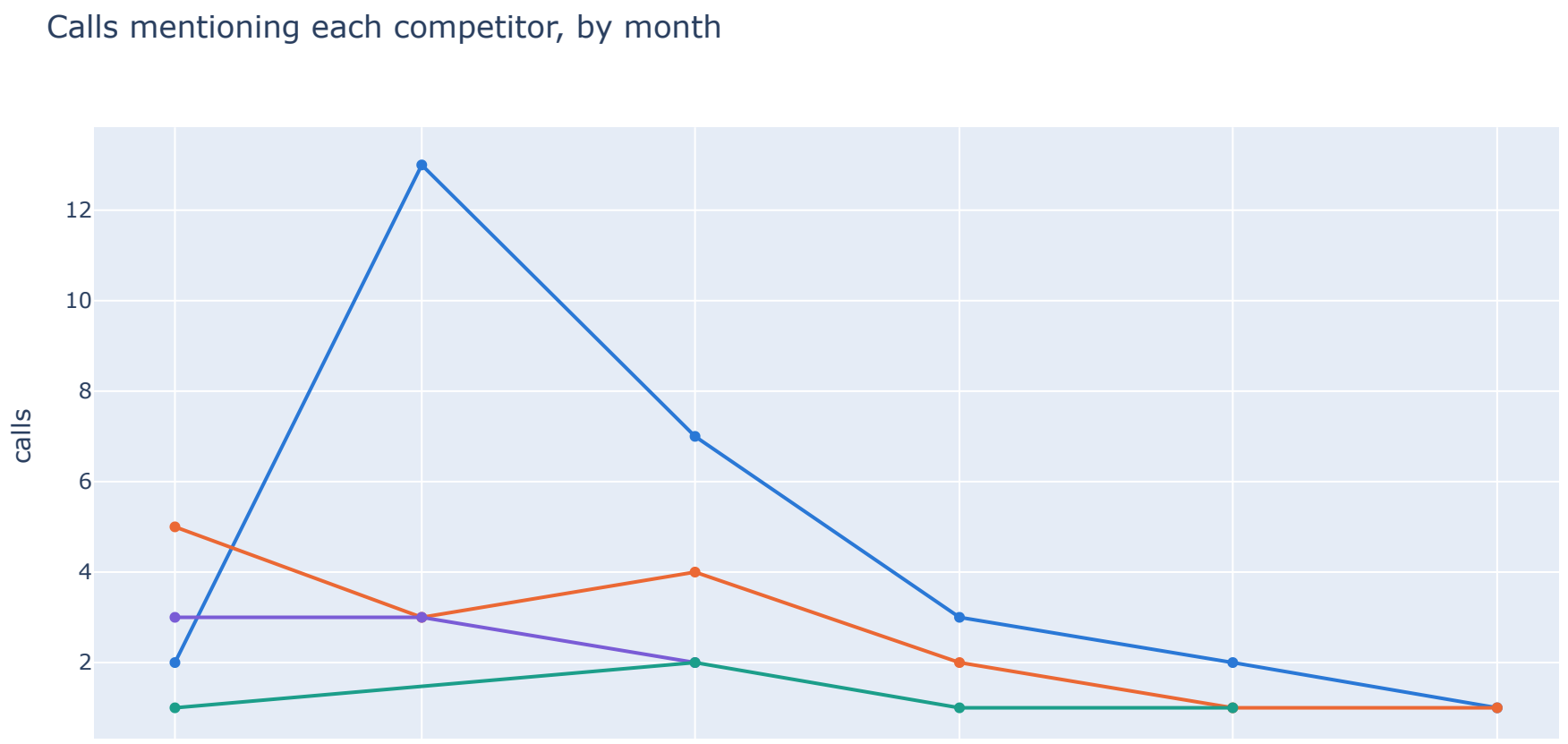
In [22]:
```
reasons = comp.explode("reasons").groupby(["competitor", "reasons"]).size().rename("mentions").reset_index()
fig = px.bar(reasons, y="competitor", x="mentions", color="reasons", orientation="h",
             category_orders={"competitor": summary.index.tolist(),
                              "reasons": list(COMPETITOR_REASONS) + ["Other"]},
```

```
            color_discrete_sequence=["#2a78d6", "#eb6834", "#7a5cd6", "#1c9e8a", "#8a8f98"],
            title="Why each competitor comes up (reason found in the sentence or its neighbours)")
fig.update_layout(yaxis_title=None, bargap=0.3)
fig
```



In [23]:
```
monthly = comp.groupby(["month", "competitor"])["call_id"].nunique().rename("calls").reset_index()
px.line(monthly, x="month", y="calls", color="competitor", markers=True,
        category_orders={"competitor": summary.index.tolist()},
```

```
        color_discrete_sequence=["#2a78d6", "#eb6834", "#7a5cd6", "#1c9e8a"],
        title="Calls mentioning each competitor, by month")
```



In [24]:
```
# What customers actually say (customer side only)
comp[comp["speaker_side"] == "customer"].assign(date=lambda d: d["start_time"].dt.strftime("%m/%d/%Y")) \
    [["date", "account", "competitor", "reasons", "quote"]].tail(10).reset_index(drop=True)
```

Out[24]:

	date	account	competitor	reasons	quote
0	04/22/2026	Brightpath	SentinelShield	[Reliability / uptime]	Honestly? It accelerated it. We had some low-level grumblings about Aegis Detect before March — the alert fatigue issue I mentioned back...
1	04/22/2026	Brightpath	SentinelShield	[Other]	Sure. So we've been talking to SentinelShield, we've had a couple of pretty deep demo sessions with them, and their threat monitoring st...
2	04/28/2026	Atlas	SentinelShield	[Pricing]	Flat pricing, I like the sound of that. Yeah, send it over and I'll take a look. I don't anticipate any major issues — honestly at this ...
3	05/06/2026	Brightpath	SentinelShield	[Reliability / uptime]	Good, because the decision is in about ten days and right now SentinelShield is ahead on our scorecard.
4	05/08/2026	Northstar	SentinelShield	[Other]	And our renewal is in June, and after March my CISO already has SentinelShield on the shortlist. This can't be another fire drill.
5	05/14/2026	Summit	VaultEdge	[Pricing]	Let's talk about the commercial side, then, because VaultEdge came in about eighteen percent under your renewal quote.
6	05/14/2026	Summit	VaultEdge	[Other]	Right now the committee is leaning toward VaultEdge for identity, and possibly for monitoring too.
7	05/14/2026	Summit	VaultEdge	[Pricing]	Flat pricing is still above VaultEdge.
8	07/01/2026	Larkspur	SentinelShield	[Reliability / uptime, Features]	We looked hard at SentinelShield and VaultEdge. Honestly, on features alone it was close.
9	07/01/2026	Larkspur	VaultEdge	[Reliability / uptime, Features]	We looked hard at SentinelShield and VaultEdge. Honestly, on features alone it was close.

What this shows.

- **SentinelShield is the main threat**, named on nearly twice as many calls as the next competitor, VaultEdge. Its matched reasons are mostly **pricing and reliability**. That pairing matters: the outage gave customers a reliability reason to look, and SentinelShield is answering with price.
- **Competitor talk peaks in March**, with the outage, and about three in four mentions on customer accounts come from accounts the outage affected (section 3).

- **Most mentions come from our own staff**, often on internal calls about positioning. The customer-side quotes above are the ones Sales should read first.
- **"Other" is still common** for internal mentions, where the reason is a product comparison the regex doesn't capture. Claude could classify reasons too, but the keyword version is enough to rank them and is free.

6. Talk time: does it matter who does the talking?

Stakeholder: Sales and support managers.

For each customer-facing call (external and support), the customer's share of speaking time is compared with the call's confidence score. Calls are split into quartiles of customer share, since most calls sit in a narrow 40–45% band and fixed cut-offs would leave some buckets empty.

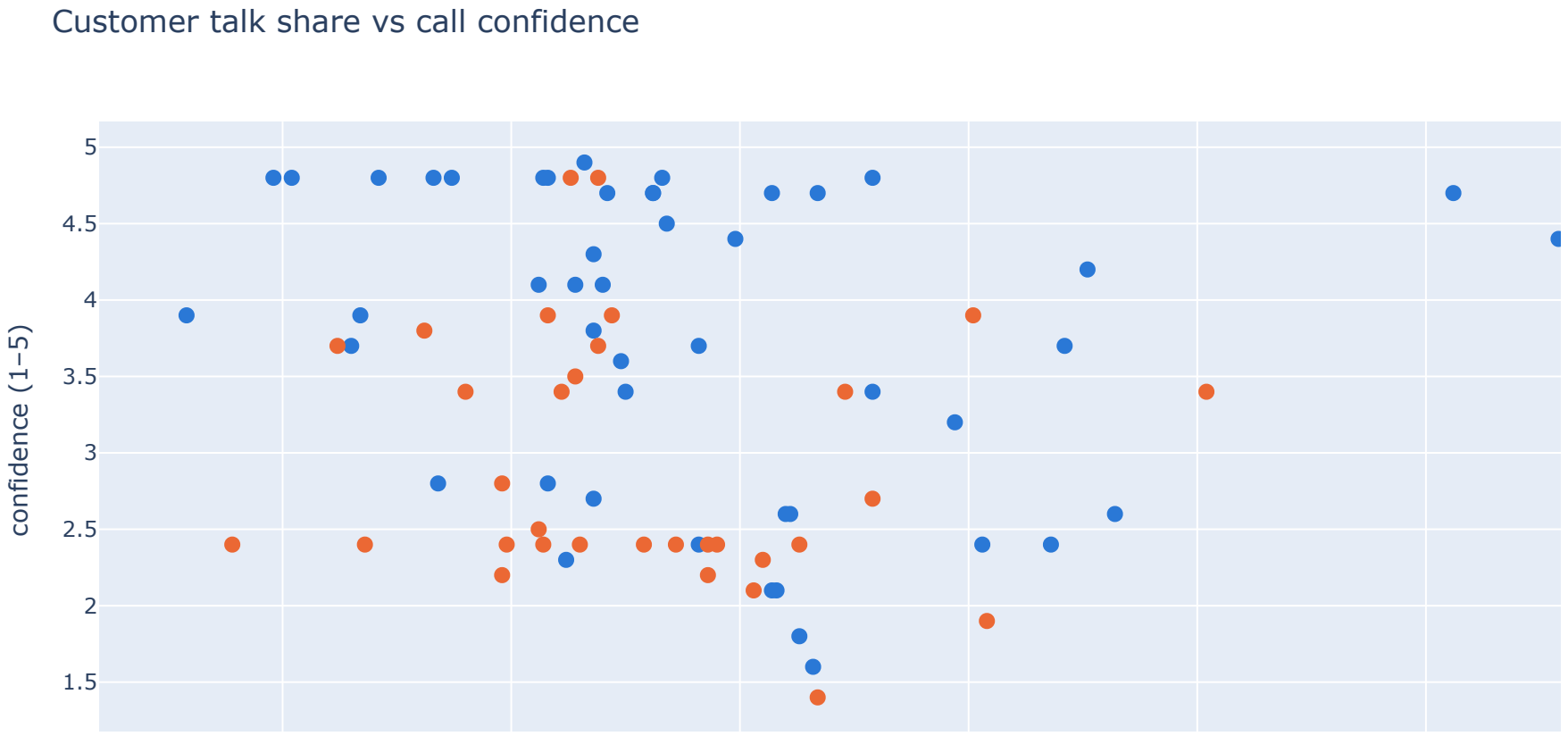
In [25]:
```
talk = hyb["talk_time"]
print(f"Correlation between customer talk share and confidence: r = {hyb['summary']['talk_share_vs_score_r']:.2f}")
talk.groupby("talk_bucket", observed=True)["sentiment_score"].agg(["mean", "count"]).round(2)
```

```
Correlation between customer talk share and confidence: r = -0.12
```

Out[25]:

talk_bucket	mean	count
Q1 (33%–41%)	3.58	21
Q2 (41%–42%)	3.82	19
Q3 (42%–46%)	3.14	20
Q4 (46%–63%)	3.15	20

In [26]:
```
fig = px.scatter(talk, x="customer_share", y="sentiment_score", color="call_type", color_discrete_map=TYPE_COLORS,
                 hover_name="title", hover_data=["lead_rep", "account"],
                 labels={"customer_share": "customer share of talk time", "sentiment_score": "confidence (1–5)"},
                 title="Customer talk share vs call confidence")
fig.update_xaxes(tickformat=".0%")
fig.update_traces(marker_size=9)
fig
```



In [27]:
```
hyb["rep_talk"].rename(columns={"lead_rep": "lead (most talk time on our side)"})
```

Out[27]:

	lead (most talk time on our side)	n_calls	avg_customer_share	avg_score
0	Maria Santos	11	0.441	3.164
1	Sarah Chen	11	0.423	3.100
2	Elena Vasquez	9	0.445	2.700
3	David Kim	8	0.425	2.637
4	Aisha Johnson	7	0.445	4.057
5	Daniel Okafor	6	0.444	3.850
6	Rachel Torres	6	0.453	3.883
7	Kevin O'Brien	5	0.388	4.620
8	Marcus Williams	4	0.443	2.525
9	Priya Patel	3	0.422	3.200
10	Lisa Park	3	0.511	4.200
11	James Mitchell	2	0.393	4.000
12	Ananya Sharma	2	0.406	4.450
13	Sofia Petrov	2	0.446	4.250
14	Ravi Gupta	1	0.441	3.700

What this shows. When customers do more of the talking, calls go worse: the two upper quartiles (customer share above about 42%) average about 3.2, against 3.6–3.8 for the two lower ones. The link is weak call by call (the correlation is only about −0.1), so it's a pattern across many calls, not a rule for any single one. The arrow points the *other* way from the usual sales advice ("let the customer talk"): here, a customer talking a lot is usually a customer **complaining**, during escalations and support cases. So this is a signal for triage, not a coaching rule.

The per-rep table is **not** a performance ranking. Reps who handle escalations and support cases (David Kim, Elena Vasquez) will average lower scores because of the calls they're given, and most reps lead fewer than ten calls. It's useful for spotting who is carrying the hard calls.

7. Action items: who is carrying the follow-up?

Stakeholder: Ops and Support leads.

Each call has vendor-extracted action items with an owner. Two views: workload per owner, and accounts whose commitments keep coming back across several calls.

In [28]:
```
wl = hyb["owner_workload"]
print(f"{wl['n_items'].sum()} action items across {len(wl)} owners; "
      f"top 5 owners hold {wl.head(5)['n_items'].sum() / wl['n_items'].sum():.0%}")
wl.head(12)
```

```
466 action items across 66 owners; top 5 owners hold 30%
```

Out[28]:

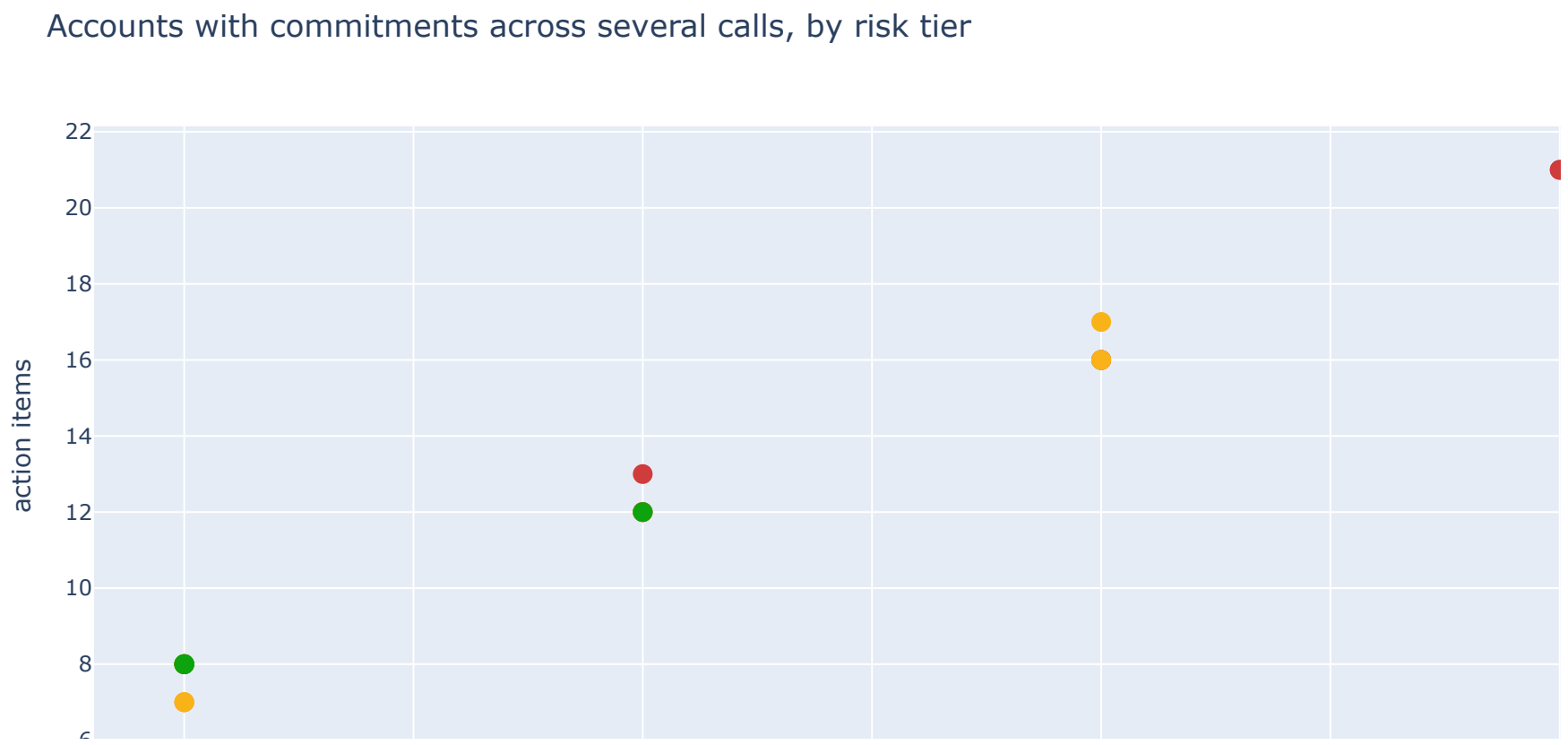
	owner	n_items	n_calls	n_accounts	share_customer
0	Maria Santos	33	15	12	0.94
1	David Kim	28	9	7	1.00
2	Sarah Chen	27	11	9	1.00
3	Elena Vasquez	27	10	7	1.00
4	Kevin O'Brien	23	13	10	0.91
5	Aisha Johnson	21	10	7	0.90
6	Lisa Park	21	11	8	0.95
7	Marcus Williams	20	10	8	0.90
8	Rachel Torres	16	9	7	0.94
9	Priya Patel	16	7	5	0.94
10	Ananya Sharma	15	13	4	0.33
11	Daniel Okafor	14	8	6	0.93

In [29]:
```
rc = hyb["recurring_commitments"].merge(health[["account", "risk_tier", "risk_score"]], on="account", how="left")
rc.head(12)
```

Out[29]:

	account	n_items	n_calls	owners	risk_tier	risk_score
0	Brightpath	21	5	[Aisha Johnson, David Kim, James Mitchell, Kendra Wallace, Marcus Williams, Owen Pritchard, Ravi Gupta, Sarah Chen]	High	65
1	Silverline	21	5	[David Kim, Dominic Flores, Kevin O'Brien, Michelle Tan, Sarah Chen]	High	76
2	Crestline	17	4	[Derek Owens, Maria Santos, Rachel Torres, Sarah Chen, Sofia Petrov]	Medium	45
3	Blackridge	16	4	[Aisha Johnson, Ananya Sharma, Lisa Park, Marcus Williams, Maria Santos, Priya Patel, Warren Beck]	Medium	38
4	Coastal	16	4	[Aegis Engineering, Elena Vasquez, Kevin O'Brien, Marcus Williams, Maria Santos, Vanessa Cruz]	High	55
5	Summit	16	4	[Alicia Monroe, David Kim, Gregory Fisk, Kevin O'Brien, Maria Santos, Sarah Chen]	High	88
6	Vanta	16	4	[Aisha Johnson, Lisa Park, Priya Patel, Raymond Burke, Sarah Chen]	Medium	45
7	Northstar	13	3	[Elena Vasquez, Maria Santos, Priya Patel]	High	65
8	Forge	12	3	[Ananya Sharma, Dana Moretti, David Kim, Elena Vasquez, Gerald Hutchins, Maria Santos]	Medium	37
9	Frostbyte	12	3	[Ananya Sharma, Elena Vasquez, Olivia Strand, Rachel Torres, Sofia Petrov, Victor Zheng]	Low	16
10	Meridian	12	3	[Jason Hoffman, Lisa Park, Marcus Williams, Rachel Torres, Ravi Gupta, Sarah Chen, Suki Nakamura]	High	66
11	Ridgeline	12	3	[Daniel Okafor, Elena Vasquez, Paula Schneider, Priya Patel, Sarah Chen]	High	71

In [30]:
```
px.scatter(rc, x="n_calls", y="n_items", color="risk_tier", color_discrete_map=TIER_COLORS, hover_name="account",
           category_orders={"risk_tier": ["High", "Medium", "Low"]},
           labels={"n_calls": "calls with action items", "n_items": "action items"},
           title="Accounts with commitments across several calls, by risk tier").update_traces(marker_size=11)
```



What this shows.

- **Follow-up is concentrated.** Five customer-facing staff own nearly a third of all action items, nearly all on customer calls. If one of them is out, commitments to several accounts slip at once.
- **High-risk accounts are the ones with the most recurring commitments.** Brightpath, Silverline and Summit each have items spread over four or five calls and owned by five to eight different people. Commitments with that many owners are the ones most likely to be dropped, which is why the account brief in the dashboard lists them in one place.

- **The data can't say what was completed.** Action items come from the call where they were agreed; nothing records whether they were done.

8. Summary: who should do what

Stakeholder	Insight	Suggested action
Customer Success / Sales	10 of 33 accounts are high churn risk, each with several signals at once; most have a renewal in play	Executive outreach to the top accounts before renewal, working from the dashboard's account briefs
Engineering / execs	The outage breached the March SLA, customers found it as early as we did, and affected accounts haven't recovered	Proactive incident detection and customer notification; follow up on SLA credits
Product	Compliance reporting and mapping across frameworks is the most widely requested capability (over half of accounts); several asks are about seeing problems sooner	Prioritise cross-framework mapping in Comply v2; in-product pipeline health and proactive alerting for Detect
Sales / Product marketing	SentinelShield leads, on price and reliability, concentrated in outage-affected accounts	Reliability proof points and a pricing response for affected accounts
Sales / support managers	Calls where customers talk more go worse: a triage signal, not a coaching rule	Route long customer monologues on support calls to escalation sooner
Ops / Support leads	Follow-up is concentrated in a few owners; high-risk accounts carry the most recurring commitments	Single owner per at-risk account; review open commitments weekly

Question	Answer
Approach	One function per stakeholder view in `src/insights.py`, built on the topic and sentiment outputs; incidents in `src/incidents.py`
Why transparent rules	No churn outcomes to train on, and an account manager needs to see why an account is flagged
LLM use	Only to name feature-gap clusters (cached). Every other insight is deterministic
How much to trust it	Risk tiers barely change between Hybrid and Rules only; the score was rebuilt signal by signal above; caveats are listed per section

Limitations.

- No CRM, ticketing or renewal data. Risk can't be checked against real churn, "resolved" depends on a call saying so, and action-item completion is unknown.
- Risk weights and tier cut-offs are judgement calls, kept in one place (`RISK_WEIGHTS` , `RISK_TIERS`) so they're easy to change.
- Competitor reasons are keyword matches and miss subtler comparisons ("Other").
- Account names come from email domains, so a customer using two domains would be split.
- With ~30 accounts and small numbers of calls per rep, the rankings are for prioritising conversations, not for statistics.

The dashboard (`streamlit run app/app.py`) shows all of this interactively in the **Accounts**, **Incidents** and **Insights** tabs.

Next: `05_dashboard.ipynb`